Eric Romero
CSC 445
4/13/2026

Homework 9 Writeup

Definitions:

- **M[i][j]: Minimum total cost for concatenating all strings from indices i to j into one string**
- **S[i][j]: Minimum cost for subarray, let's call this k**
- **sum(i, j): Total length of strings from indices i to j**

1. **Algorithm Description**

   Refer to definitions above.

   Create two tables of size n x n (for M and S)
   Handle smallest subproblems first -> Base Case:
   Assuming index i, if we check interval [i .. i], it should only have one string, so no concatenation. M[i][i] will be equal to 0

   Handle Larger subproblems in increasing order by length.
   Assuming indices i to j where i < j, We need to find the split point (we call k) where i <= k < j.
   This point splits the interval into two parts, that is, [i .. k] and [k + 1 .. j]

   If the split point is chosen, total cost is the combination of:
   1. **Cost for concatenating [i .. k]**
   2. **Cost for concatenating [k + 1 .. j]**
   3. **Cost for merging both group**

   Final Cost: Total length of strings i to j

2. **Recurrence Relation**

   Base Case: If i == j, means only one string → M[i][i] = 0 → S[i][i] = -1
   Recurrence:
   For intervals with more than one string, try every point k between indices i to j - 1.
   If split at k:
   a. **Left side: indices i to k → solved with cost M[i][k]**
   b. **Right side: indices k + 1 to j 0> solved with cost M[k + 1][j]**
   c. **Add together into single group with sum(i, j)**

   Relation is:
   M[i][j] = min(M[i][k] + m[k + 1][j] + sum(i, j)) for indices i <= k < j

3. **Correctness**

Eric Romero
CSC 445
4/13/2026

Prove: Algorithm finds minimum possible total concatenation cost for every interval in stringLengths[i .. j]

Base Case where i = j:

No concatenation. Interval is one string. Minimum cost = 0. Matches initialization: M[i][i] = 0

Recursive Case where intervals have more than one string:

Groupings are adjacent, and together cover a full interval. There exists an index k (i <= k < j) that allows a final merging between the two groups stringLengths[i .. k] and stringLengths[k + 1, j]. Both subgroups have to be concatenated into single groups before a final merge can happen.

Left Side: Concatenated with minimum possible cost -> M[i][k]

Right Side: Concatenated with minimum possible cost -> M[k+1][j]

Merging both Sides: Cost is sum(i, j), because combining all strings from indices i to j

Equation to consider: M[i][k] + M[k+1][j] + sum(i, j)

The algorithm will check all split indices k and choose the smallest value in the range of indices i <= k < j.

M[i][j] = minimum possible cost for concatenating the interval stringLengths[i .. j]

Through constructing solutions from increasing length order, the algorithm will find the optimal solution for every possible interval.

The table S[i][j] stores the split indices that find that minimum for each interval. Using recursion, we use these indices to reconstruct the optimal concatenation order. This should return a valid optimal sequence

4. **Reconstructing Optimal Solution**

Using the table S[i][j], we will use this to reconstruct the optimal concatenation solution. For each interval from indices i to k, S[i][j] is recorded being the split index k that was used to find the optimal solution.

As previously mentioned,

Base Case i == j: one string, no concatenation, no k

Recursive Case: interval[i to j] is split into a left and right side, interval[i to k] and interval[k+1 to j] respectively, where k is S[i][j].

Eric Romero
CSC 445
4/13/2026

Reconstruction finds the optimal concatenation order for the left and right subintervals, recording its respective index k. Recording after both recursive calls makes sure that we return a valid order sequence.

5. **Time and Space Complexity in terms of n**

Assume n: number of strings

We are solving all interval subproblems from indices i to j using two tables, n x n. There are O(n^2) intervals because as mentioned above, we are solving for all intervals. For each interval, there are split point k's, where in the worst case, an interval will take O(n) possible splits.

Time Complexity: O(n^3) = O(n^2) subproblems + O(n) splitting per one

Regarding space, We will use two tables using O(n^2) space each. That's the biggest worry. Using the sum array takes up O(n) and reconstructing will take O(n) as well.

Space Complexity: O(n^2)

6. **Pseudocode**

```
optimalOrder(stringLengths):
    n = length(stringLengths)

    If n < 2:
        Return null array

    Output list created - output

    Create array prefixSums[n + 1] -> prefixSums[0] = 0
    For i in 0 to n - 1:
        PrefixSums[i + 1] = prefixSums[i] + stringLengths[i]
```

Create tables M and S for storing minimal cost for interval i to j, and storing split index k that gives minimal cost respectively.

```
    M[n][n]
    S[n][n]

    // base case
    For i in 0 to n - 1
        M[i][i] = 0
        S[i][i] = -1

    // Larger Intervals
```

Eric Romero
CSC 445
4/13/2026

```
        For len in 2 to n
                For i in 0 to n - len:
                        J = i + len - 1
                        M[I][J] = INF
                        S[i][j] = -1

                        interSum = prefixSum[j + 1] - prefixSum[i]

                        For k = i to j - 1
                                candidateCost = m[i][k] + m[k+1][j] + interSum

                                If candidateCost < m[i][j]:
                                        M[i][j] = candidateCost
                                        S[i][j] = k


                bestSolution(s, 0, n - 1, output)

bestSolution(s, i, j, output)
        If i >= j:
                Return

        K = s[i][j]

        bestSolution(s, i, k , output)
        bestSolution(s, k + 1, j , output)

        Add k to output
```